

ES 336: Computer Organization and Architecture



Sameer G Kulkarni

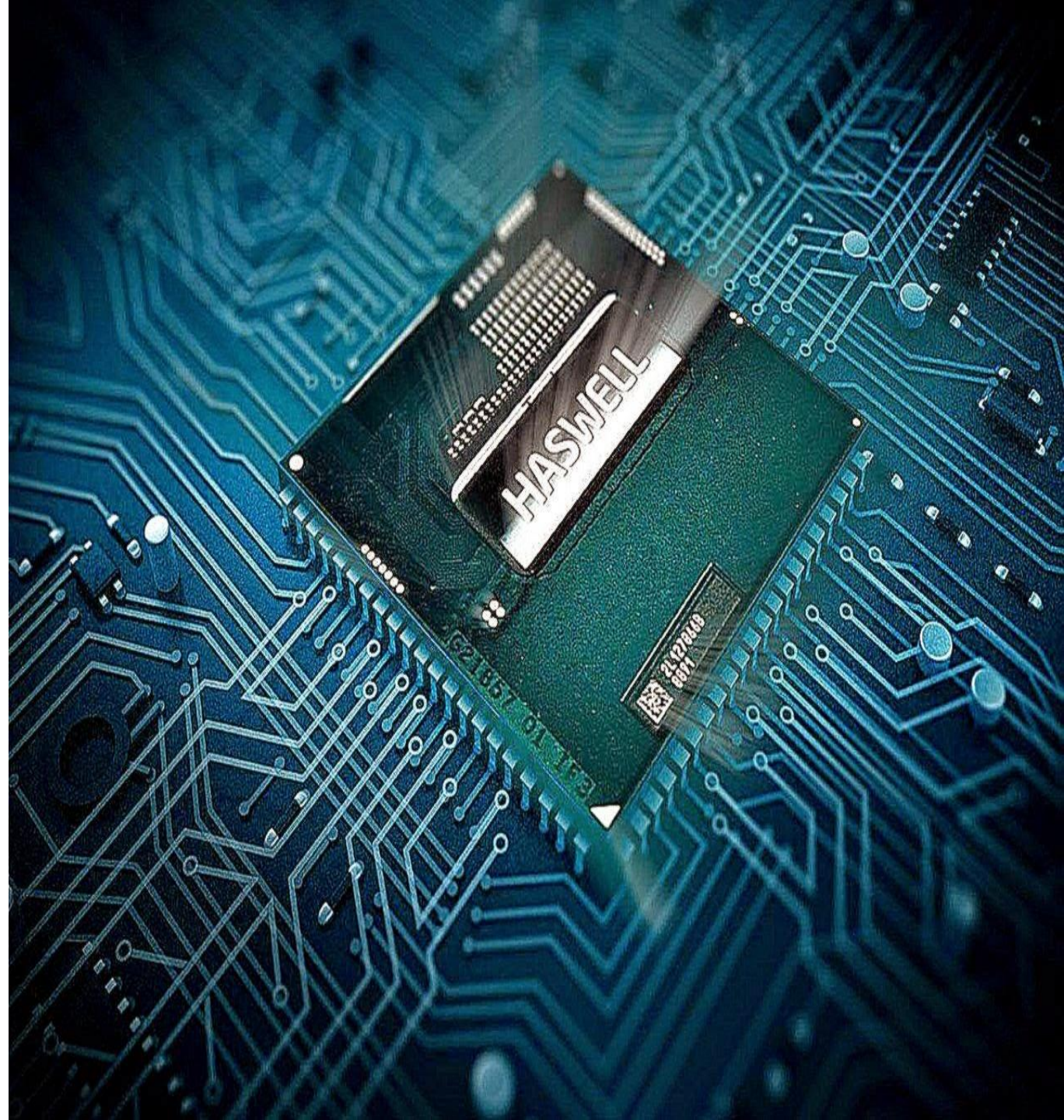
Assistant Professor,

*Department of Computer Science and Engineering,
Indian Institute of Technology, Gandhinagar*

Today's Lecture

- MIPS Processor Design
 - Basic elements of processor
 - Datapath and Control paths
- Processor Design Modes
 - Single-cycle Design
 - Multi-cycle Design
 - Pipelined Design
- Reading Materials:
 - Chapter 4 – COAHP
 - Chapters 14,15 – COAWS

Source: [Wikipedia](#)



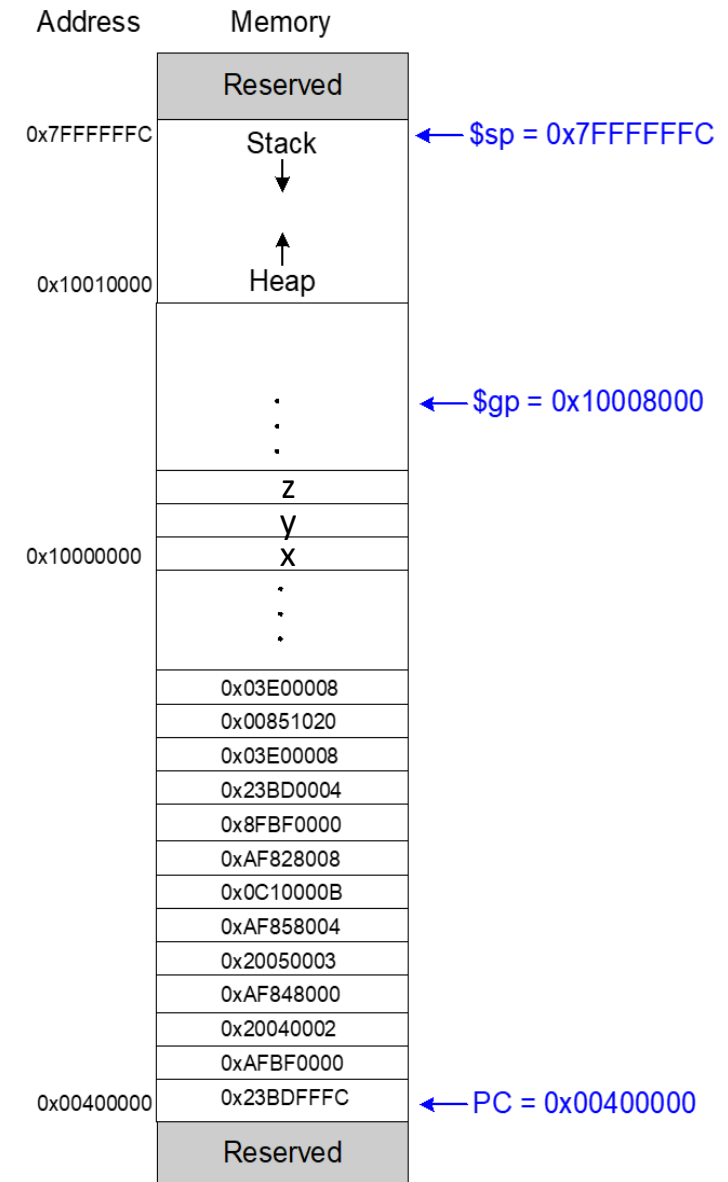
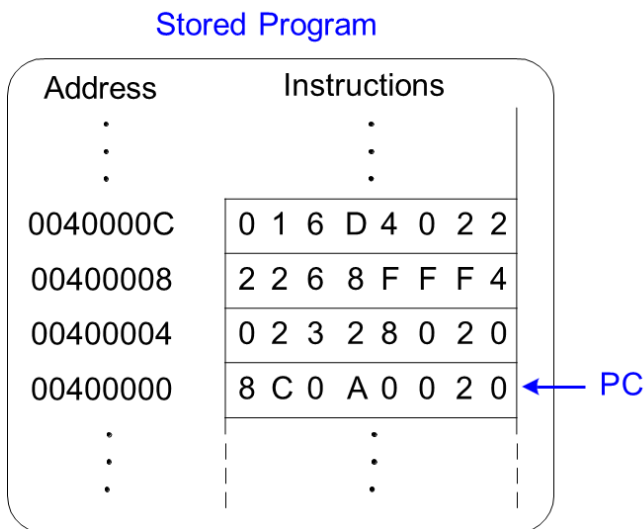
RECAP – WHAT WE HAVE COVERED SO FAR

- **Abstract Computer Model:**
 - Basic components and Interconnects.
 - Implications on computer Architecture.
 - **End-2-End working of a compute program**
 - Role of system software; Stages of compilation
 - Memory map of a program and its execution.
 - **Instruction Set Architecture**
 - What is ISA?; Importance of ISA; Design principles; ISA Categorization.
 - Assembly Language & Machine Language
 - **Associated Topics**
 - Different computing models.
 - Evolution of ISA
 - Data Endianness
- **Reading Materials:**
 - Chapter 1 – COAHP
 - Chapter 2 – COAHP
2.1-2.3, 2.5-2.9, 2.10-2.12
Not covered: 2.4, 2.11
 - Chapters 3, 12 – COAWS

MIPS ARCHITECTURAL ASPECTS

- The processor hardware executes the program:
 - fetches (reads) the instructions from memory in sequence
 - performs the specified operation
 - continues with the next instruction
- The program counter (PC) tracks the current instruction
 - In MIPS, programs typically start at memory address ***0x00400000***

Assembly Code	Machine Code
lw \$t2, 32(\$0)	0x8C0A0020
add \$s0, \$s1, \$s2	0x02328020
addi \$t0, \$s3, -12	0x2268FFF4
sub \$t0, \$t3, \$t5	0x016D4022

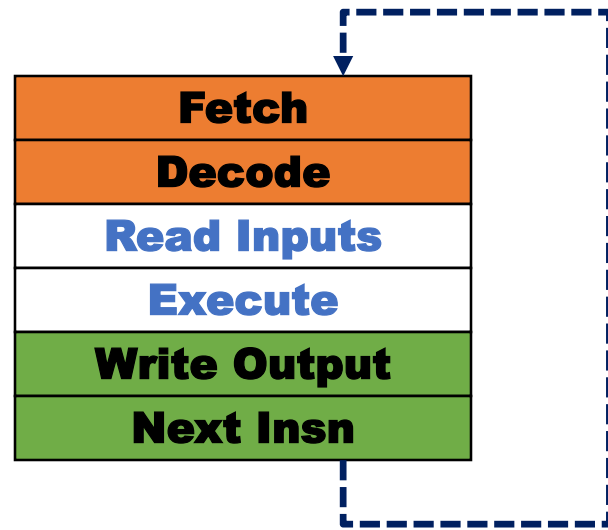
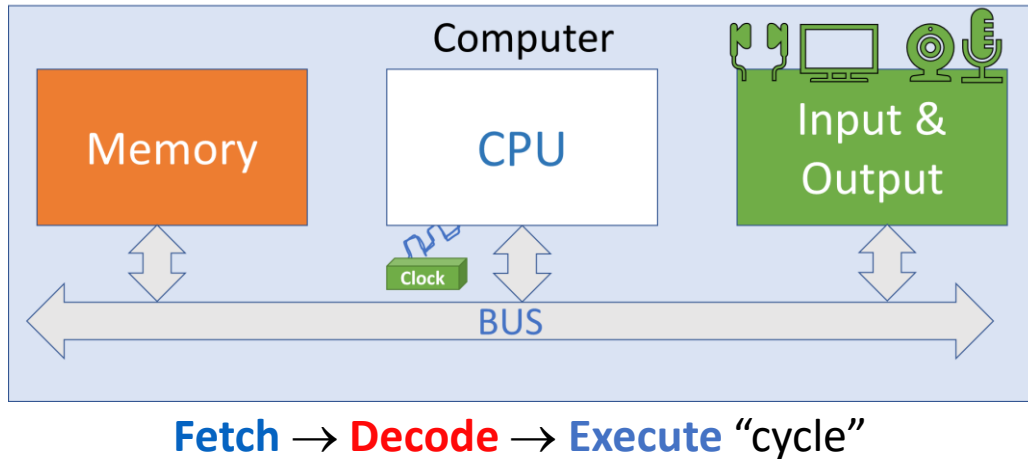


- **Microarchitecture:** *how to implement an architecture in hardware*

- Processor:
 - **Datapath:** functional blocks
 - **Control:** control signals
- Multiple implementations for a single architecture:
 - **Single-cycle:** Each instruction executes in a single cycle
 - **Multicycle:** Each instruction is broken into series of shorter steps and executed over multiple cycles.
 - **Pipelined:** Each instruction broken up into series of steps & multiple (different) instructions can execute at once.

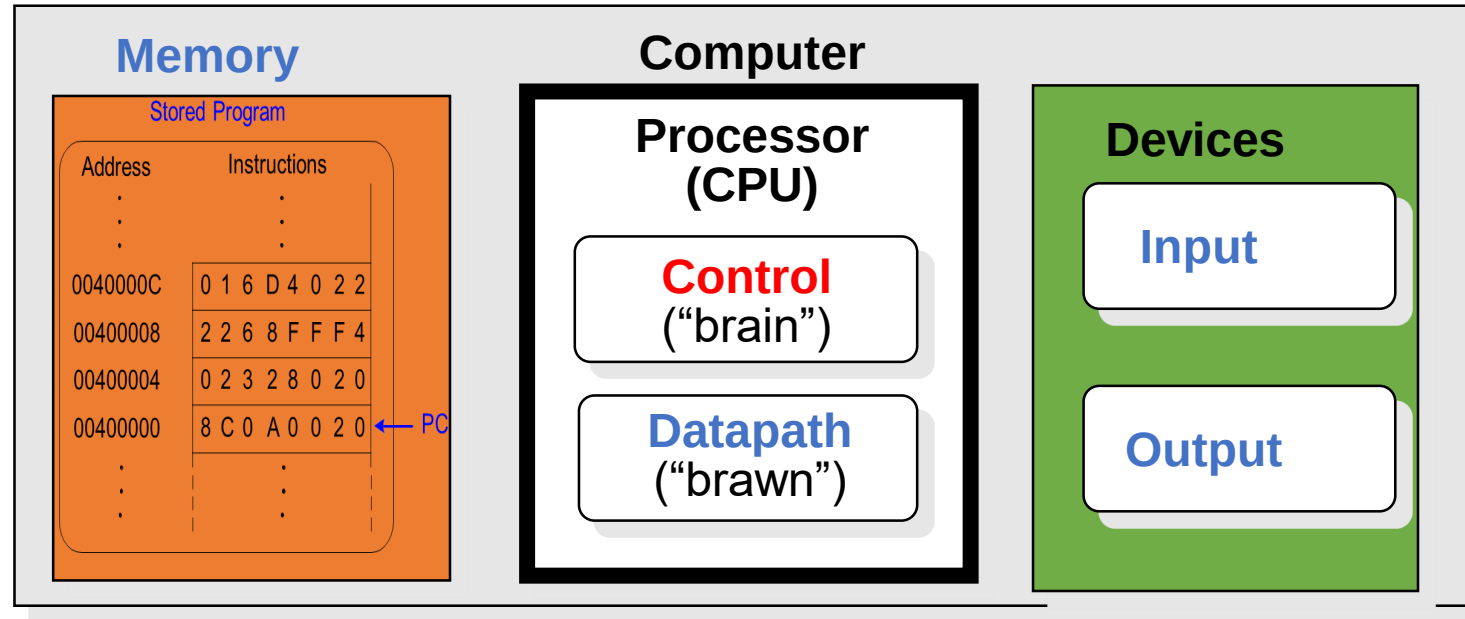
Application Software	programs
Operating Systems	device drivers
Architecture	instructions registers
Micro-architecture	datapaths controllers
Logic	adders memories
Digital Circuits	AND gates NOT gates
Analog Circuits	amplifiers filters
Devices	transistors diodes
Physics	electrons

INSTRUCTION EXECUTION MODEL



- **Program is just “data in memory”**
 - Makes computers programmable (“universal”)
- The computer is just finite state machine
 - **Registers** (few of them, but fast)
 - **Memory** (lots of memory, but slower)
 - **Program counter** (next instruction to execute)
- A computer executes **instructions**
 - **Fetches** next instruction from memory
 - **Decodes** it (figure out what it does)
 - **Reads** its **inputs** (registers & memory)
 - **Executes** it (adds, multiply, etc.)
 - Write its **outputs** (registers & memory)
 - **Next insn** (adjust the program counter)

DATA PATH AND CONTROL PATH



Fetch → Decode → Execute "cycle"

Datapath: consists of all the elements in a processor that are dedicated to storing, retrieving, and processing data such as **register files, memory, and the ALU**

Datapath performs computation

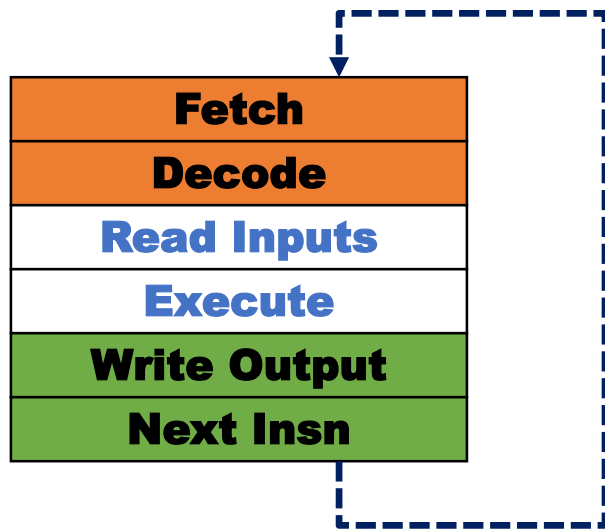
ISA specific: can implement every instruction (single-cycle: in one pass!)

Control path: control unit that generates the appropriate signals to control the movement of data and execution of instructions in the data path.

determines which computation is performed

Routes data through the data path (which regs, which ALU operation)

TYPICAL INSTRUCTION EXECUTION STEPS



Steps involved in MIPS Instruction Execution

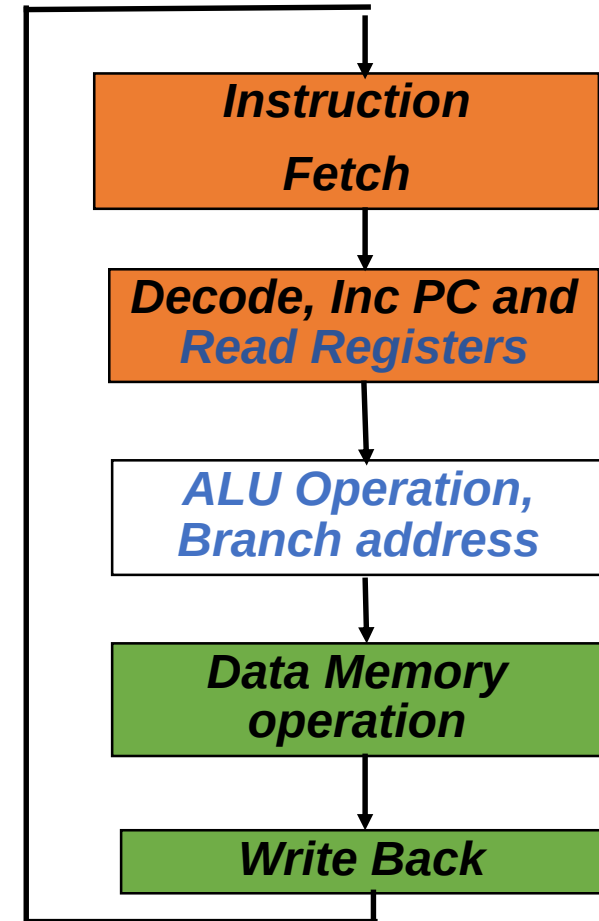
Step1: Instruction Fetch

Step2: Instruction Decode + Read Register + *Inc PC*

Step3: Execute Instruction (ALU Operation)

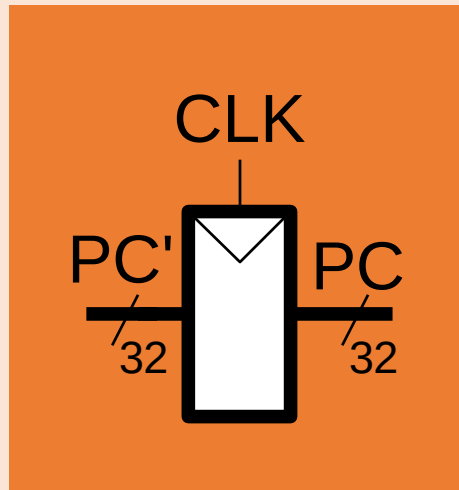
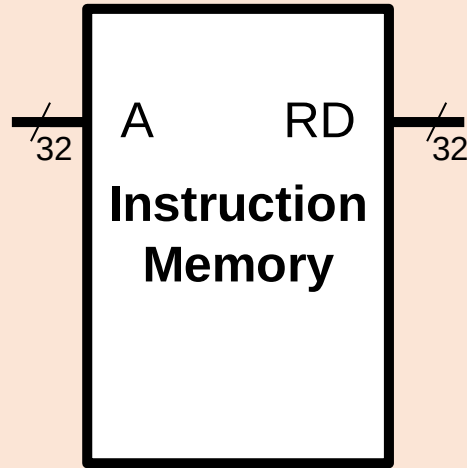
Step4: Data Memory Operation

Step5: Write back (to register)

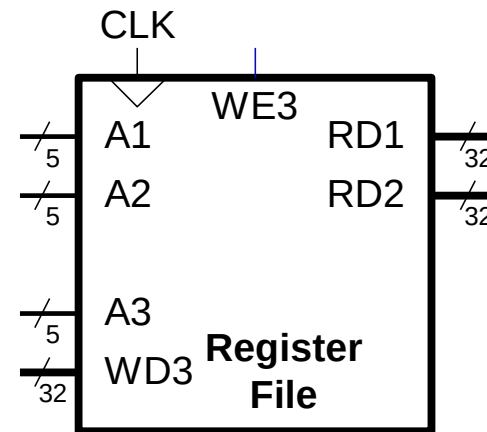
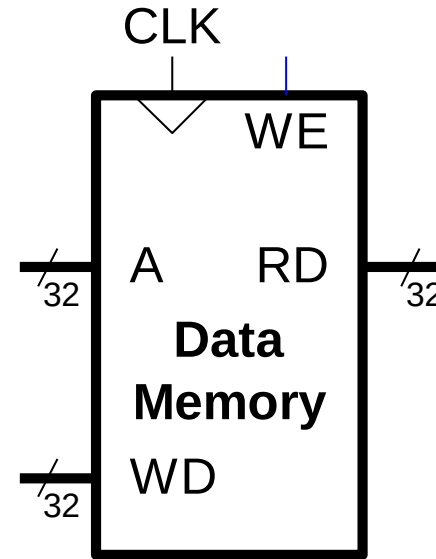


BASIC ELEMENTS OF A SIMPLE MIPS PROCESSOR DESIGN

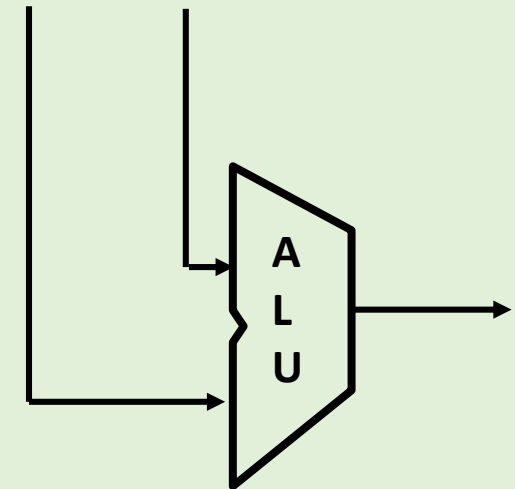
Instructions



Data/Operands

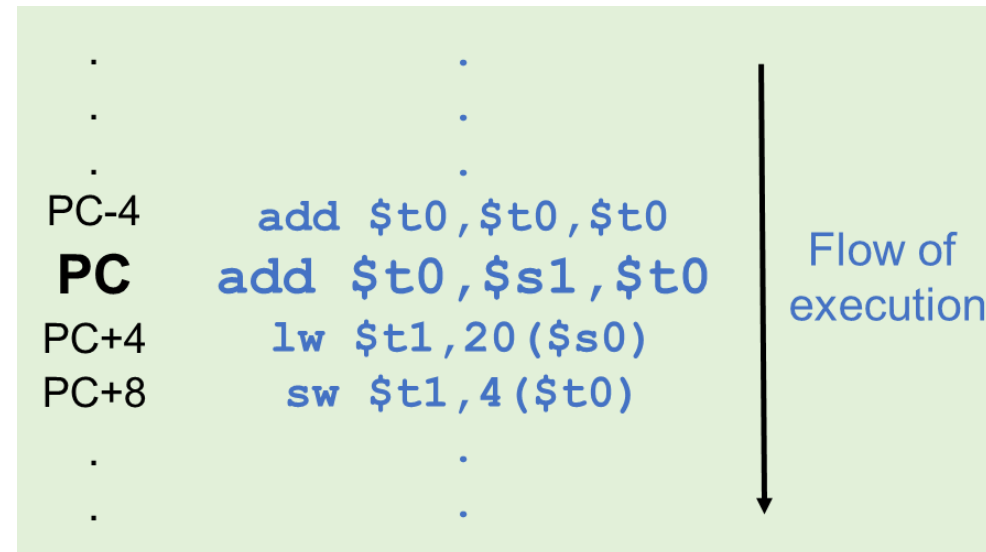
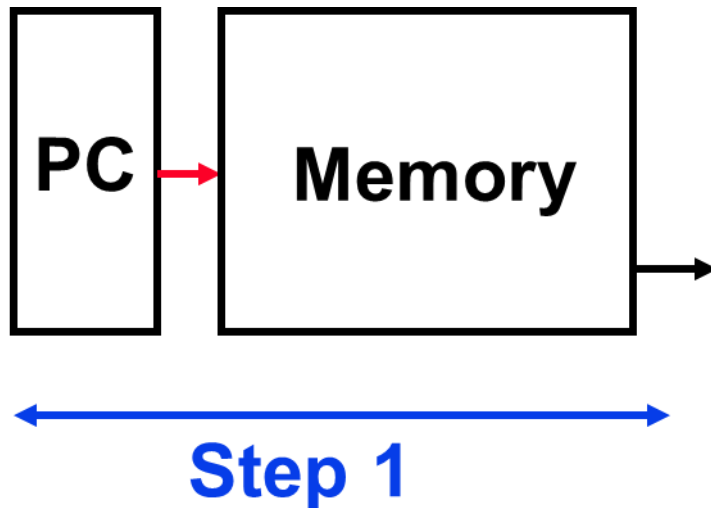
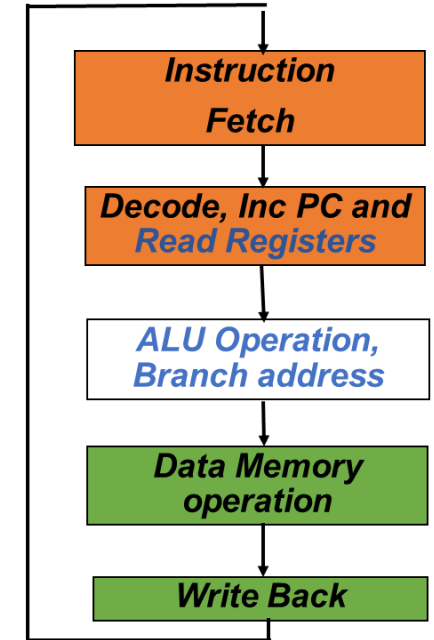


Compute

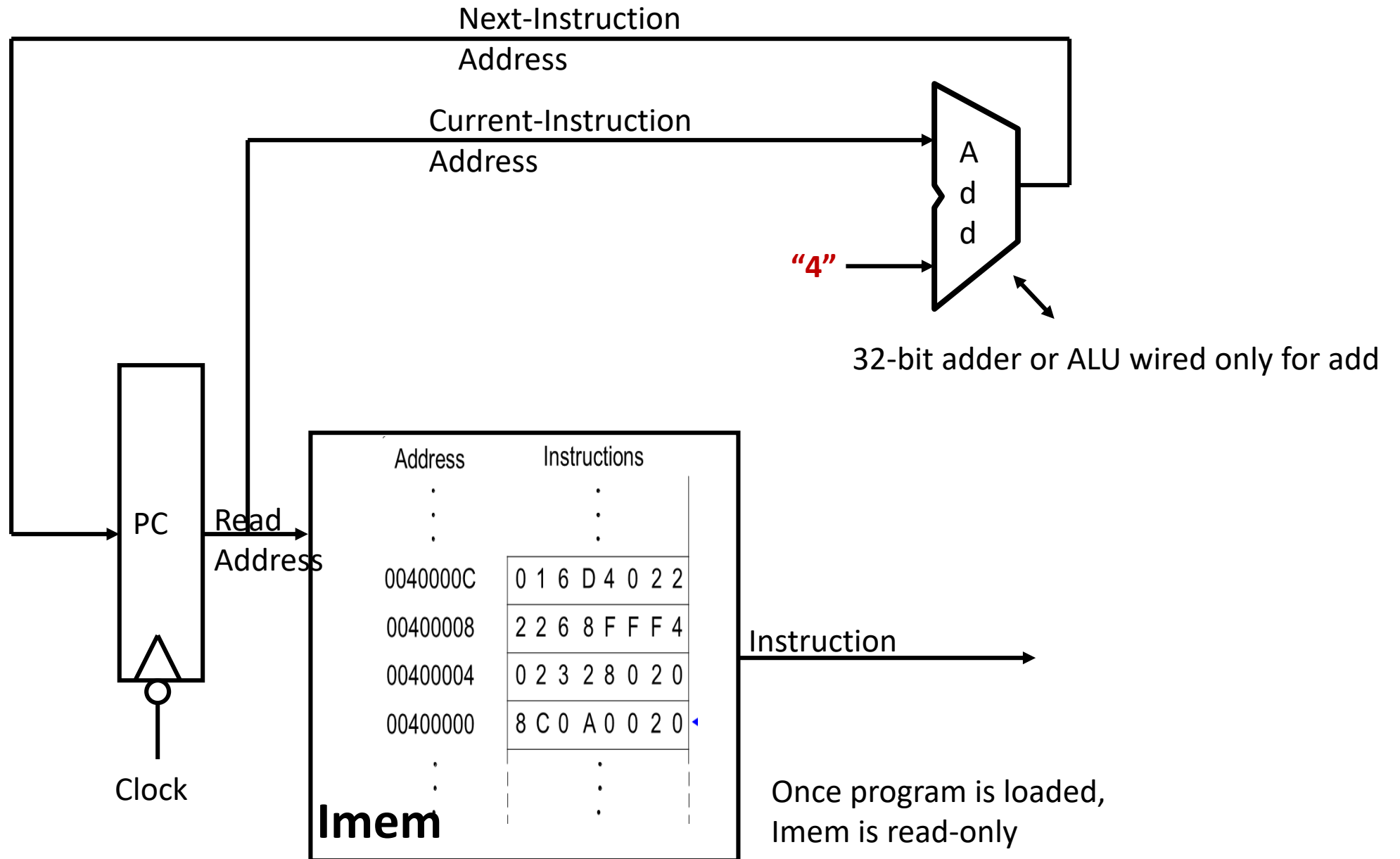


LET US BEGIN WITH SIMPLE SUBSET OF MIPS INSTRUCTIONS

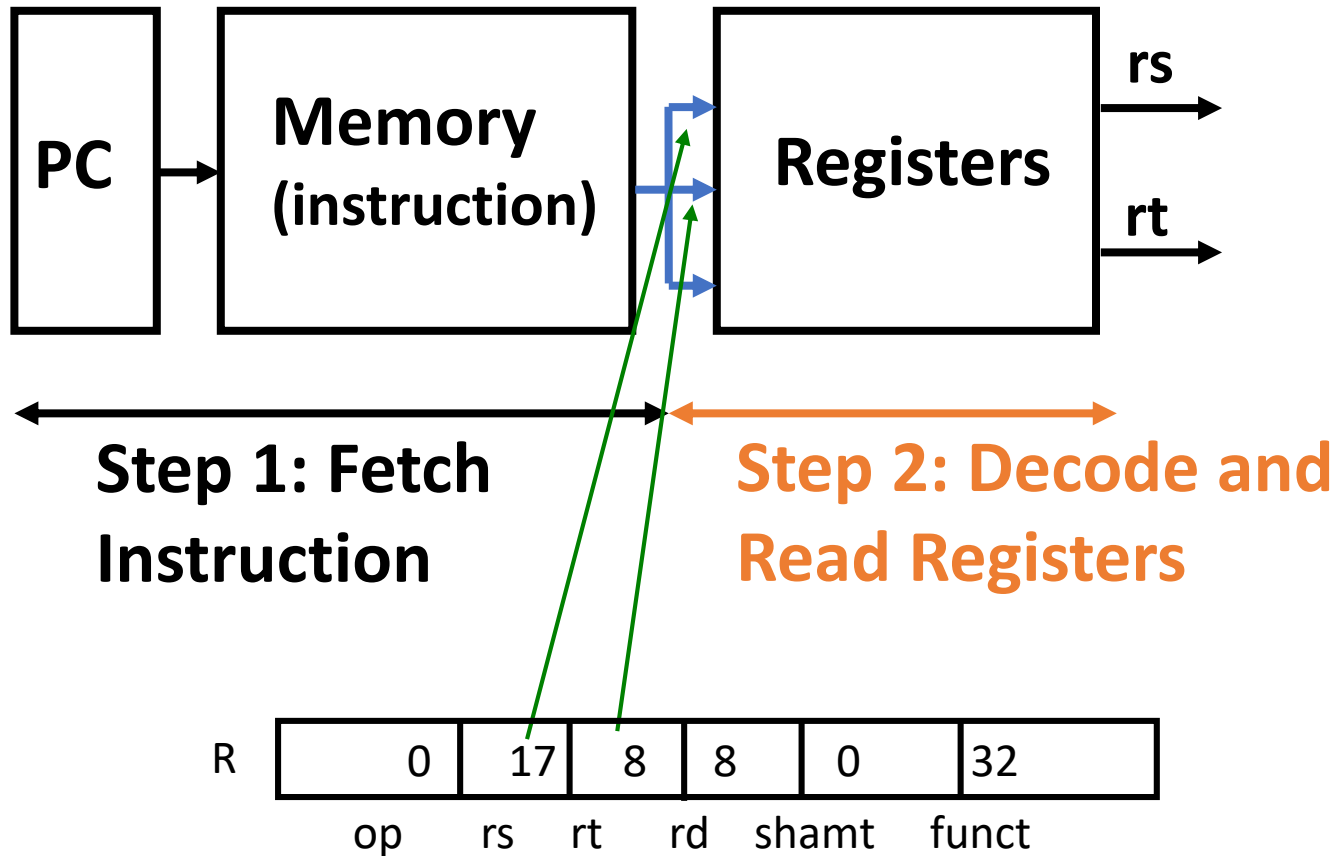
- Let us consider a subset of MIPS instructions:
 - ALU instructions: `and`, `or`, `add`, `sub`, `sllt`
 - Memory instructions: `lw`, `sw`
 - Branch instructions: `beq`, `j`
- Just enough instructions to illustrate the key ideas and basic working of MIPS ISA



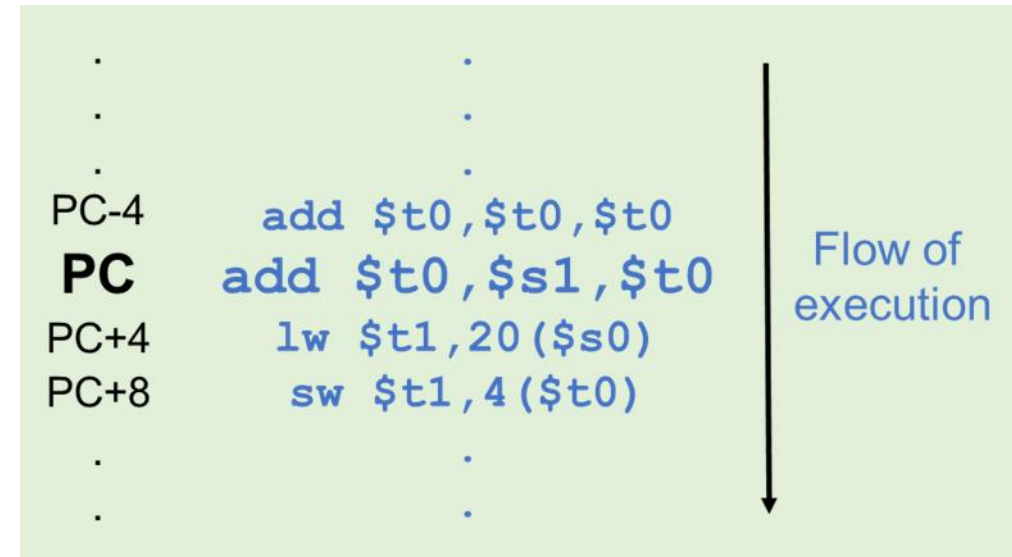
DATAPATH STEP 1 (INSTRUCTION FETCH): ANY INSTRUCTION



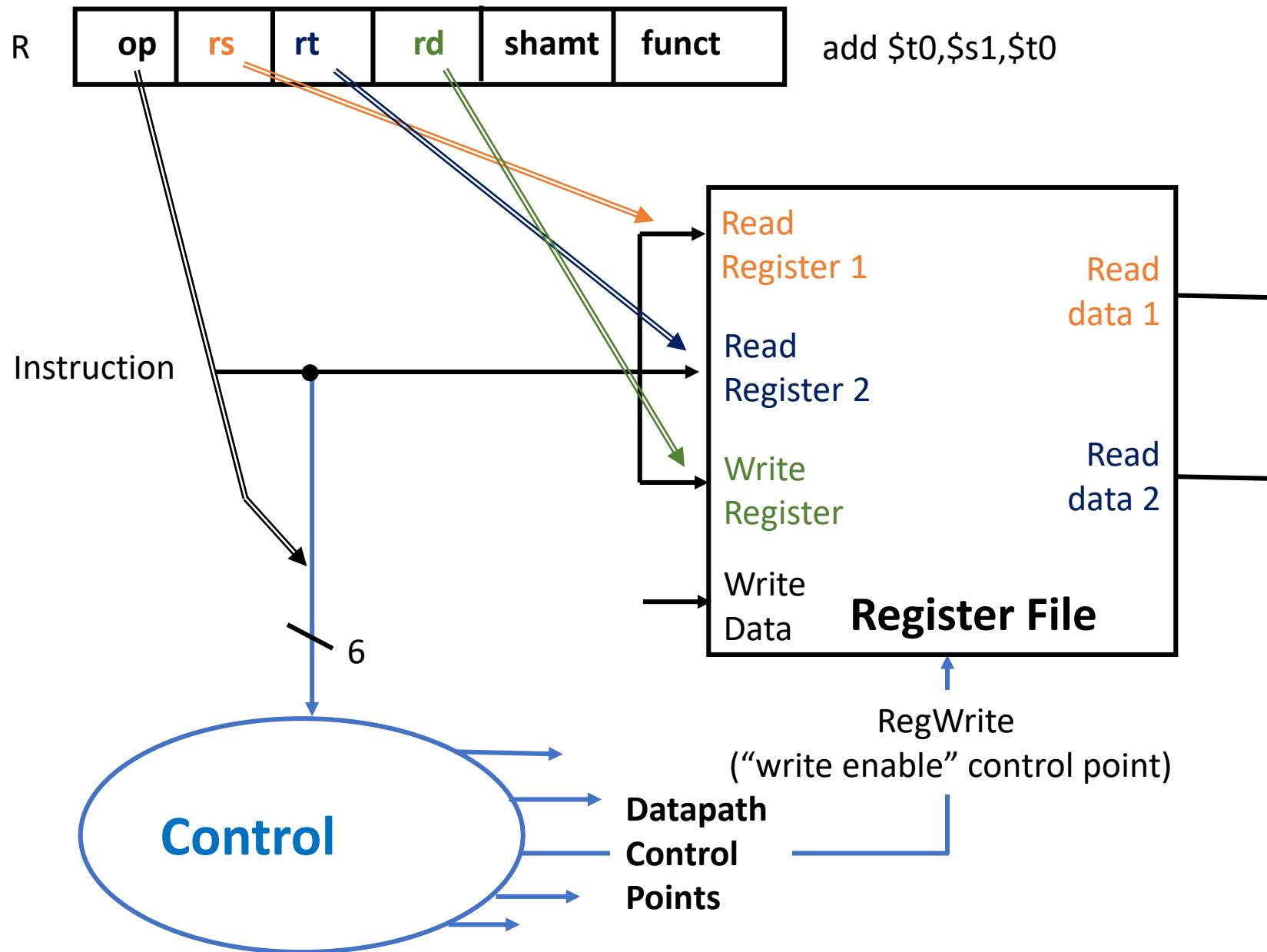
DATAPATH STEP 2: DECODE AND READ REGISTER



add \$t0,\$s1,\$t0

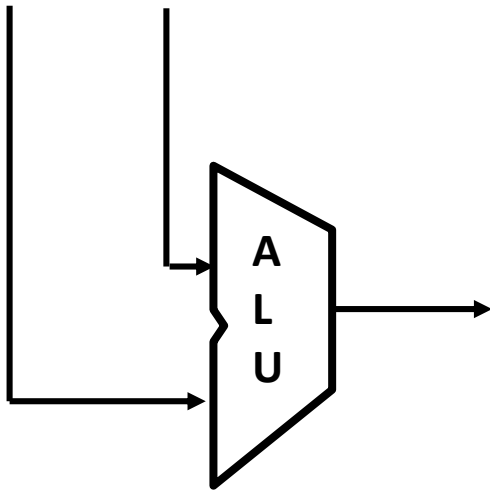


DATAPATH STEP 2 (DECODE AND READ REGISTER)



MIPS DATAPATH STEPS

- 3rd step onwards depends on instruction class
- EX: for ALU instructions, add \$t0, \$t1, \$t2 outputs from registers t1 and t2 will be sent to the ALU input.
- For Memory-reference instruction: $Address \leftarrow Base + offset$
lw \$t0, 20(\$s0)



R-Type

sub \$t0, \$s7, \$s3

op	rs	rt	rd	shamt	funct
6 bits	5 bits	5 bits	5 bits	5 bits	6 bits

I-Type

addi \$s7, \$s1, -15

op	rs	rt	imm
6 bits	5 bits	5 bits	16 bits

J-Type

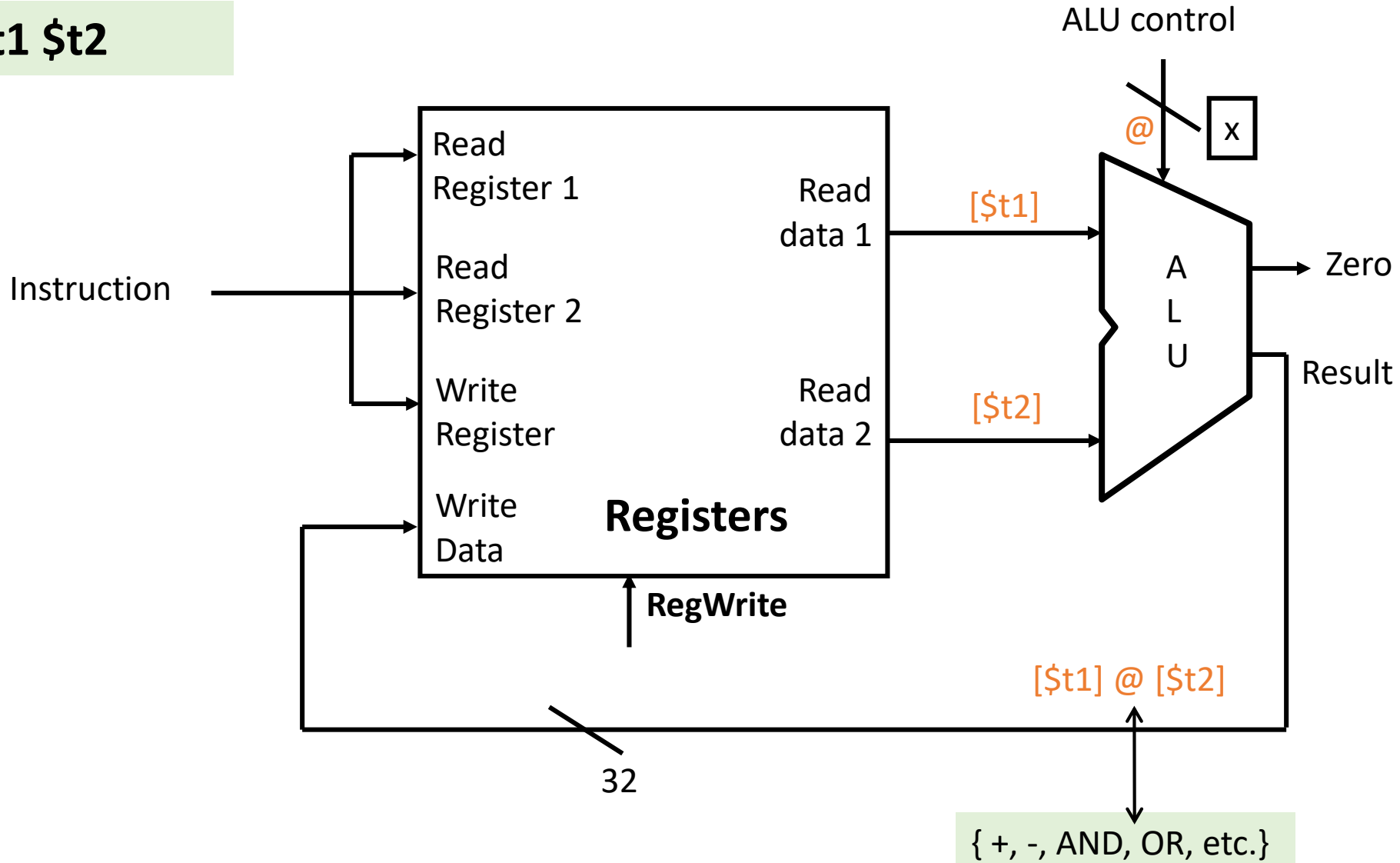
j loop

op	addr
----	------

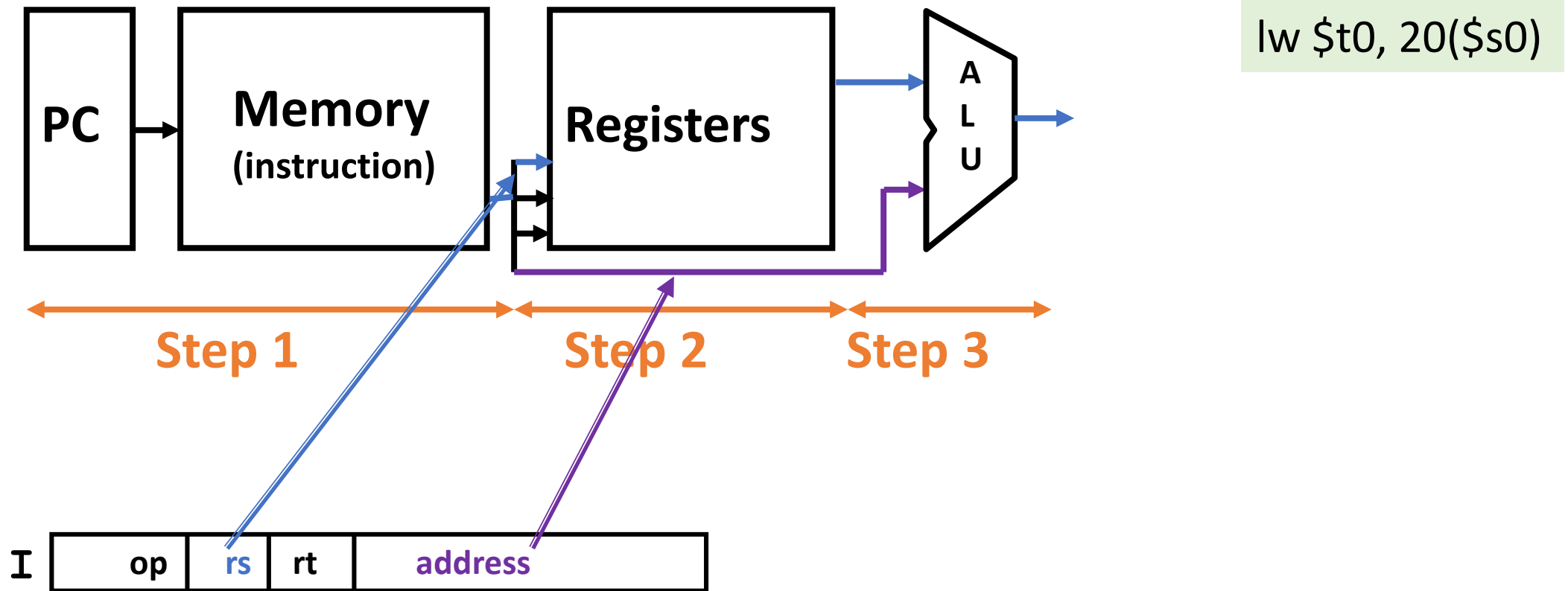
DATAPATH STEP 3: R-FORMAT INSTRUCTIONS

add, sub, and, or, slt

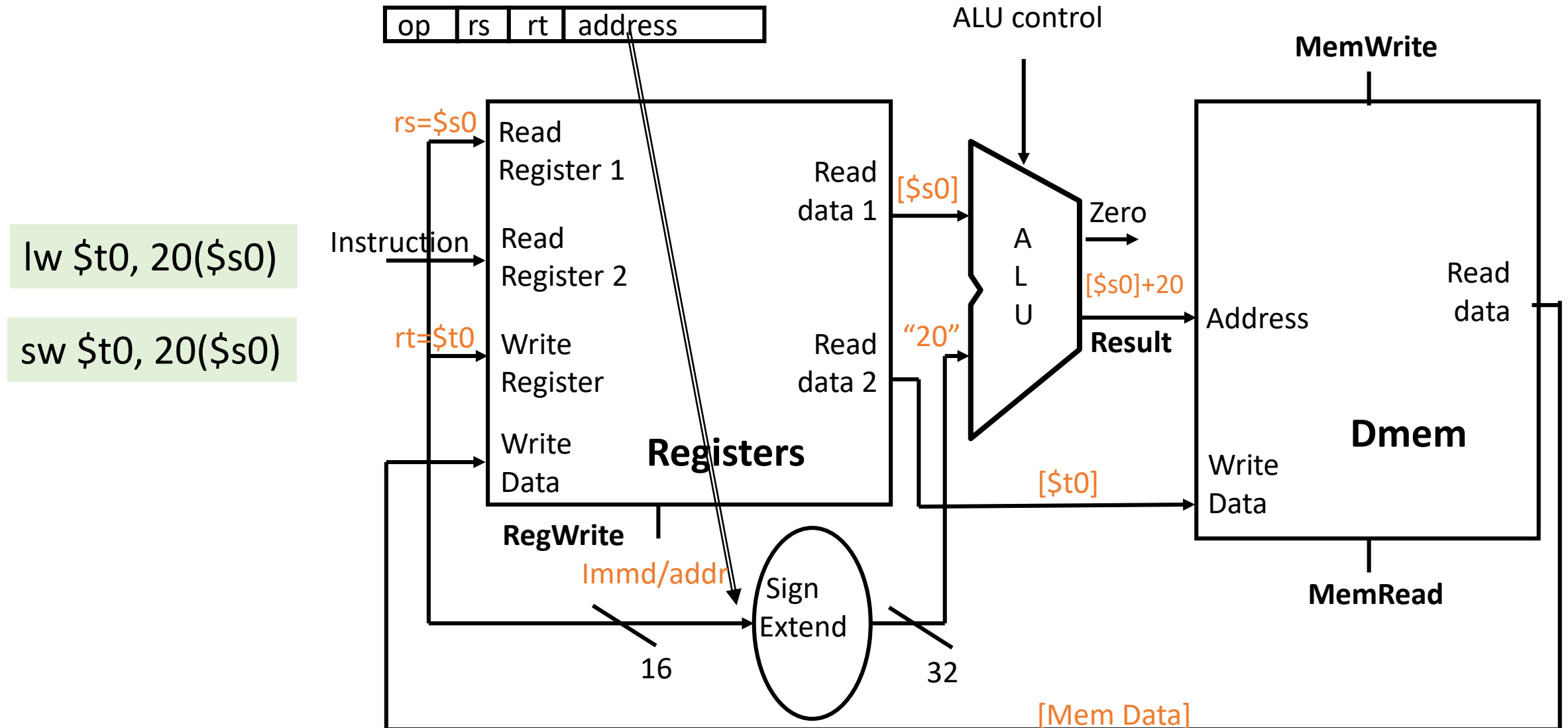
add \$t0 \$t1 \$t2



DATAPATH STEP 3: I-FORMAT (LW)

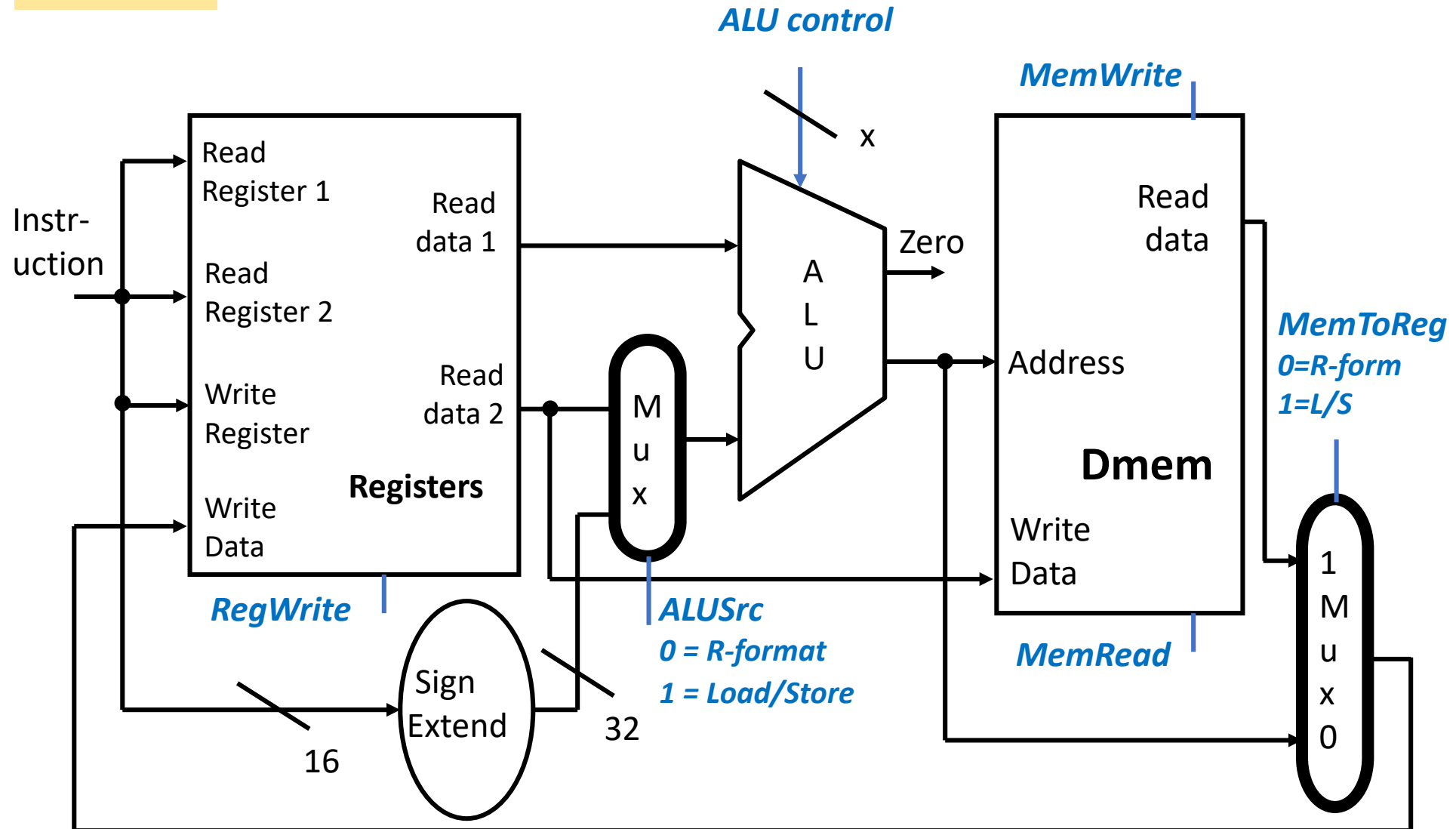


DATAPATH STEP 3-5: LOAD/STORE



COMPOSE DATAPATH: R-FORM + LOAD/STORE

Add muxes

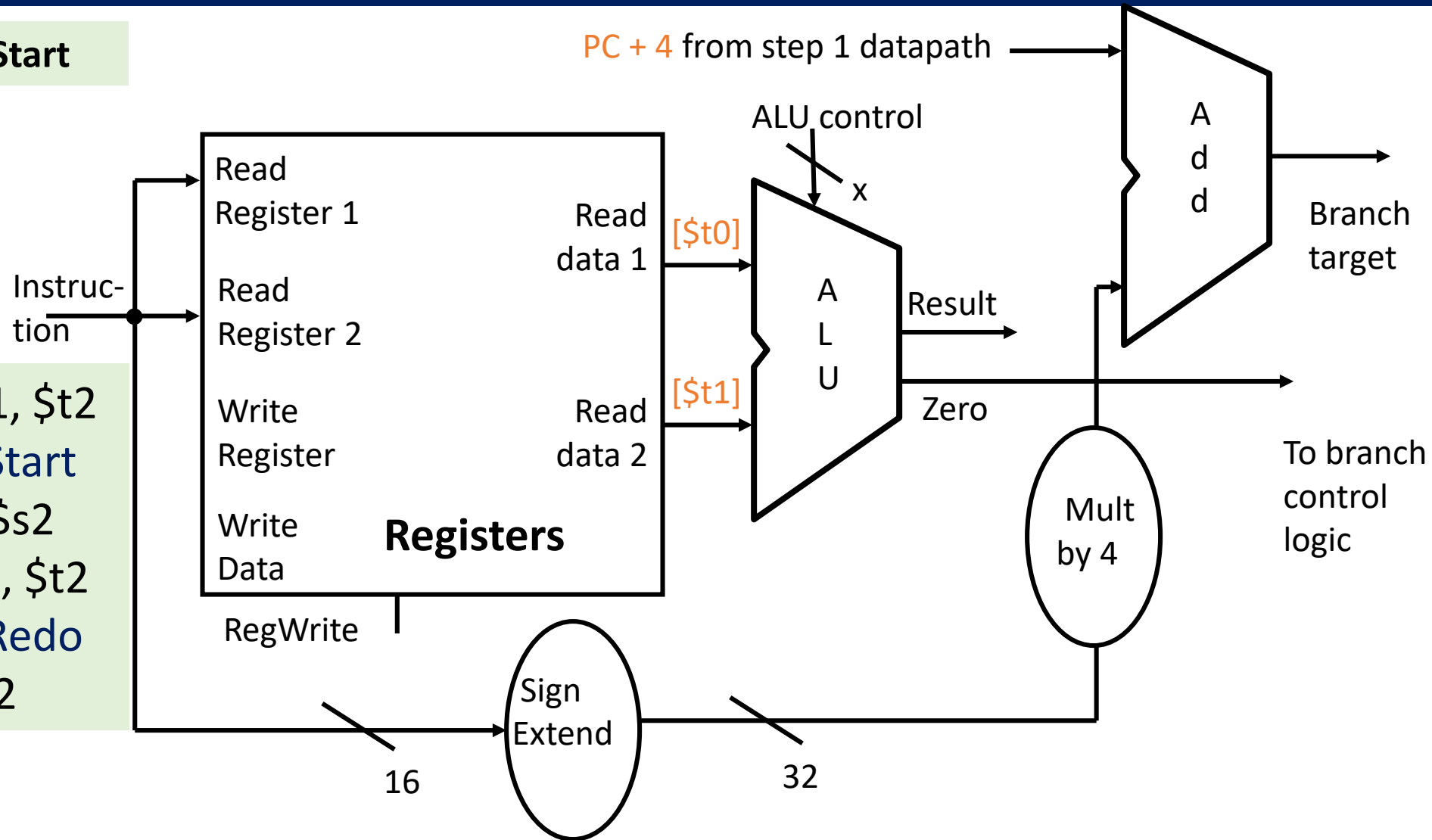


DATAPATH STEP 3: I-FORMAT (BRANCH)

beq \$t0,\$t1,Start

PC + 4 from step 1 datapath

1000: **Start:** add \$t0, \$t1, \$t2
1004: beq \$t0, \$t1, **Start**
1008: add \$s0, \$s1, \$s2
1012: **Redo:** sub \$t0, \$t1, \$t2
1016: beq \$t0, \$t1, **Redo**
1020: addi \$t2, \$t2, 2

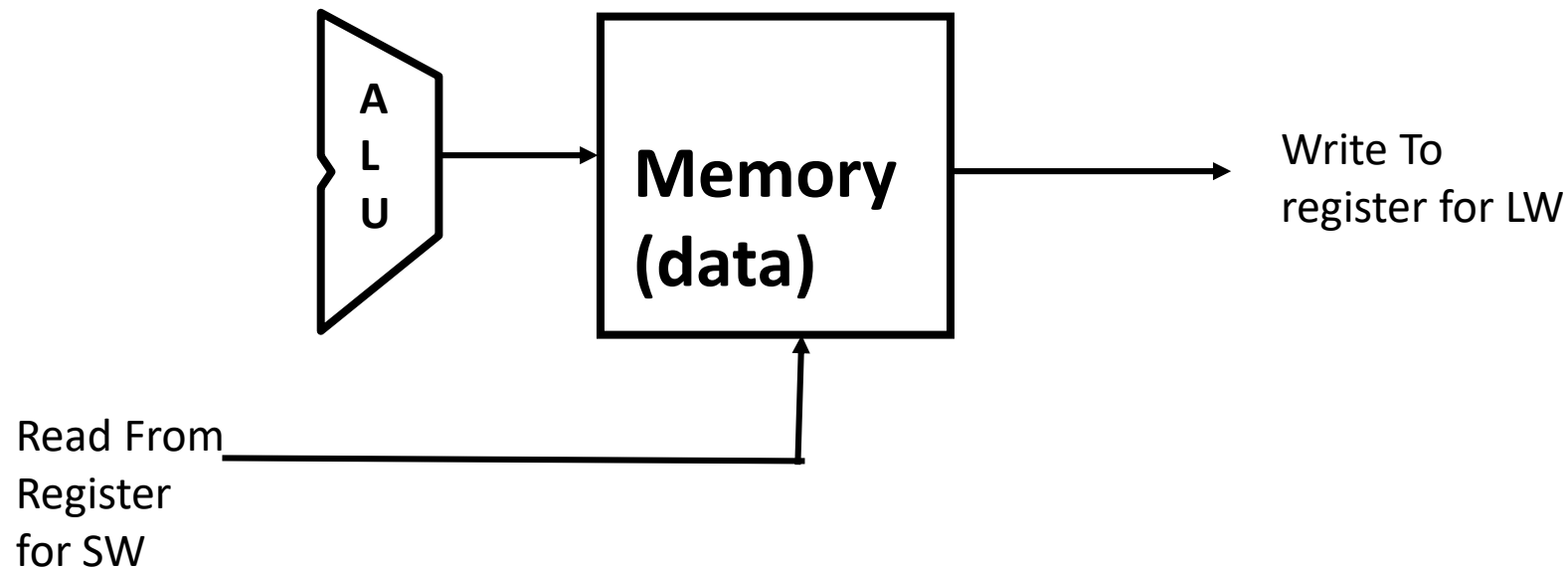


DATAPATH STEP 4: (LW/SW)

- **4th step** depends on instruction class

For LW : Fetch Data from Memory and Write to Register
 $DataReg \leftarrow Mem[Address]$

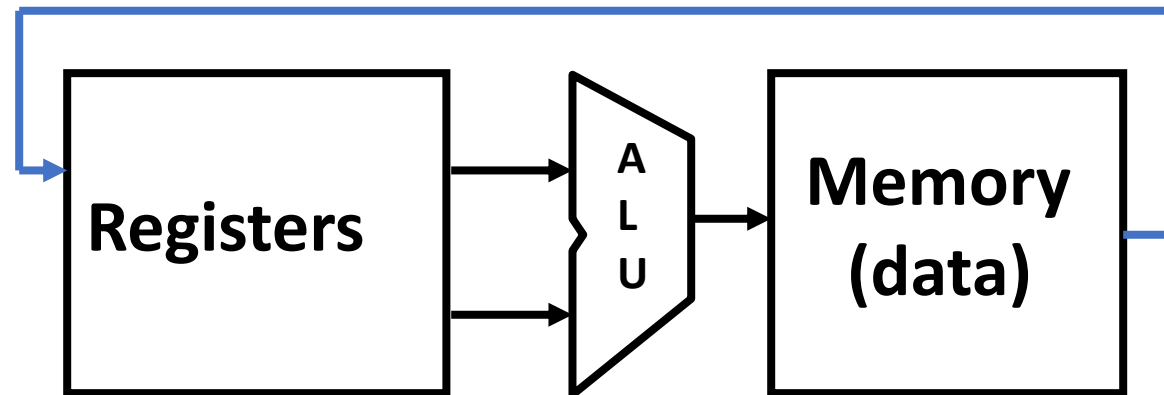
For SW : Put the contents of a register in Memory
 $Mem[Address] \leftarrow DataReg$



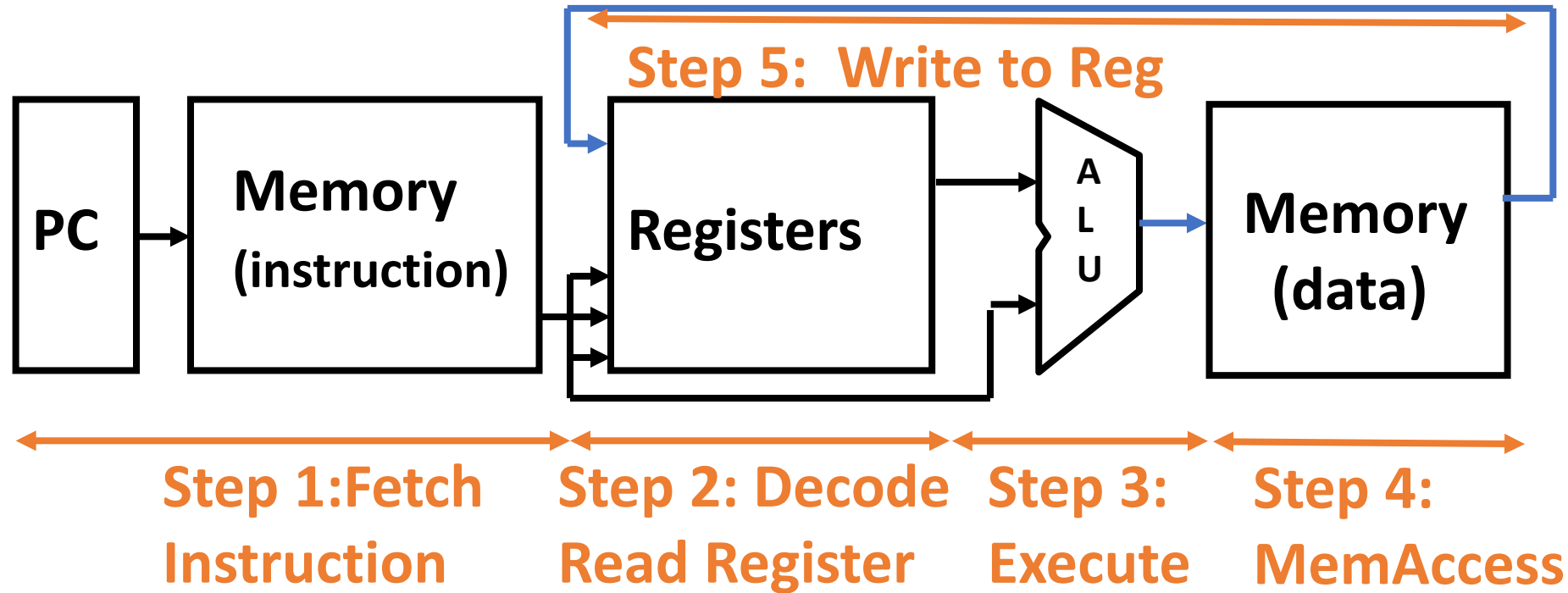
° 5th step only for `lw`; rest are done

EX: for `lw`: Write Result

$Reg[rt] \leftarrow Data$



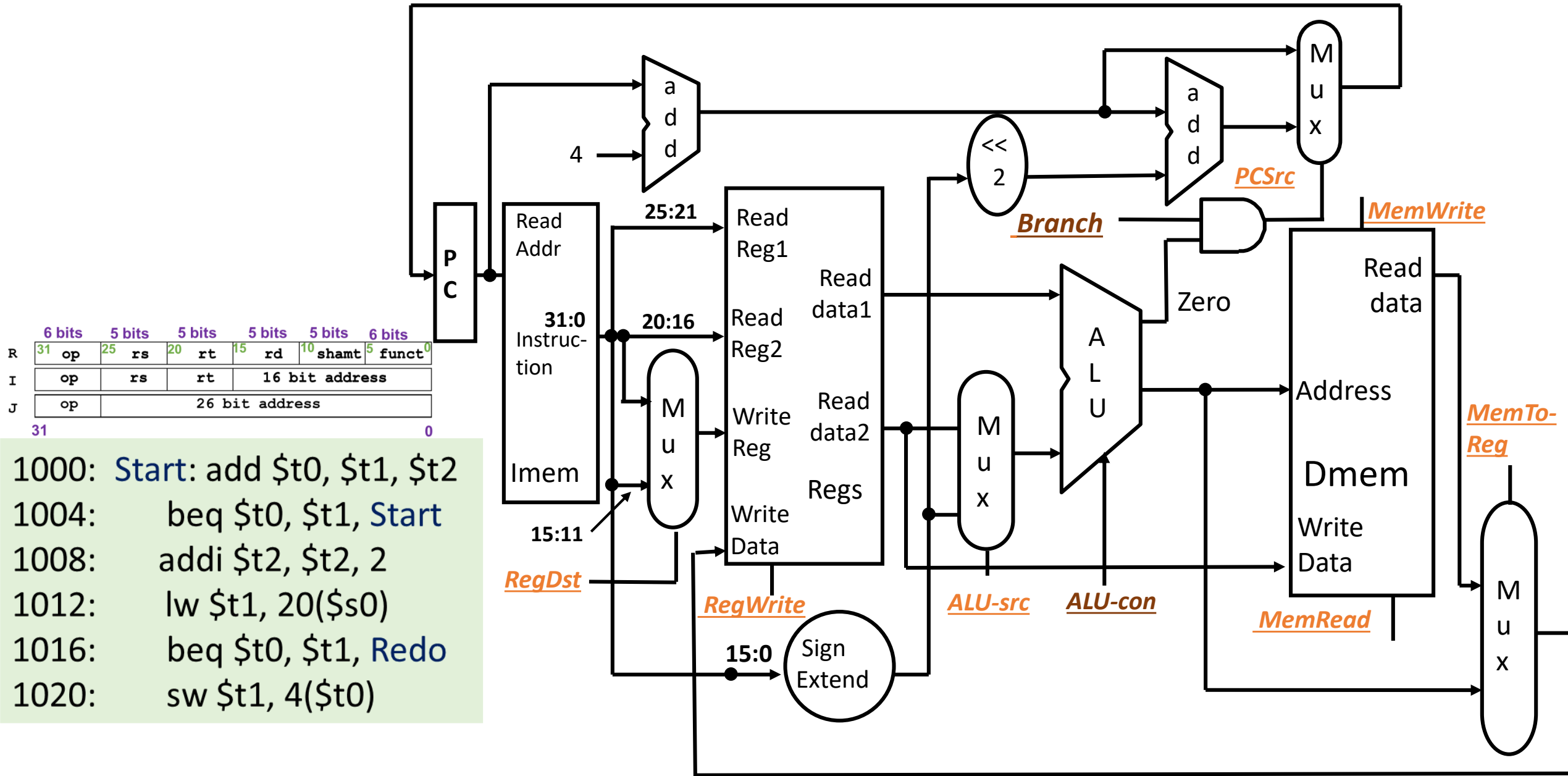
HIGH LEVEL VIEW OF 5 STEPS MIPS INSTRUCTION EXECUTION



```
1000: Start: add $t0, $t1, $t2
1004:      beq $t0, $t1, Start
1008:      addi $t2, $t2, 2
1012:      lw $t1, 20($s0)
1016:      beq $t0, $t1, Redo
1020:      sw $t1, 4($t0)
```

IF + INC PC
ID + Reg Rd
Execute
Mem Access
Writeback

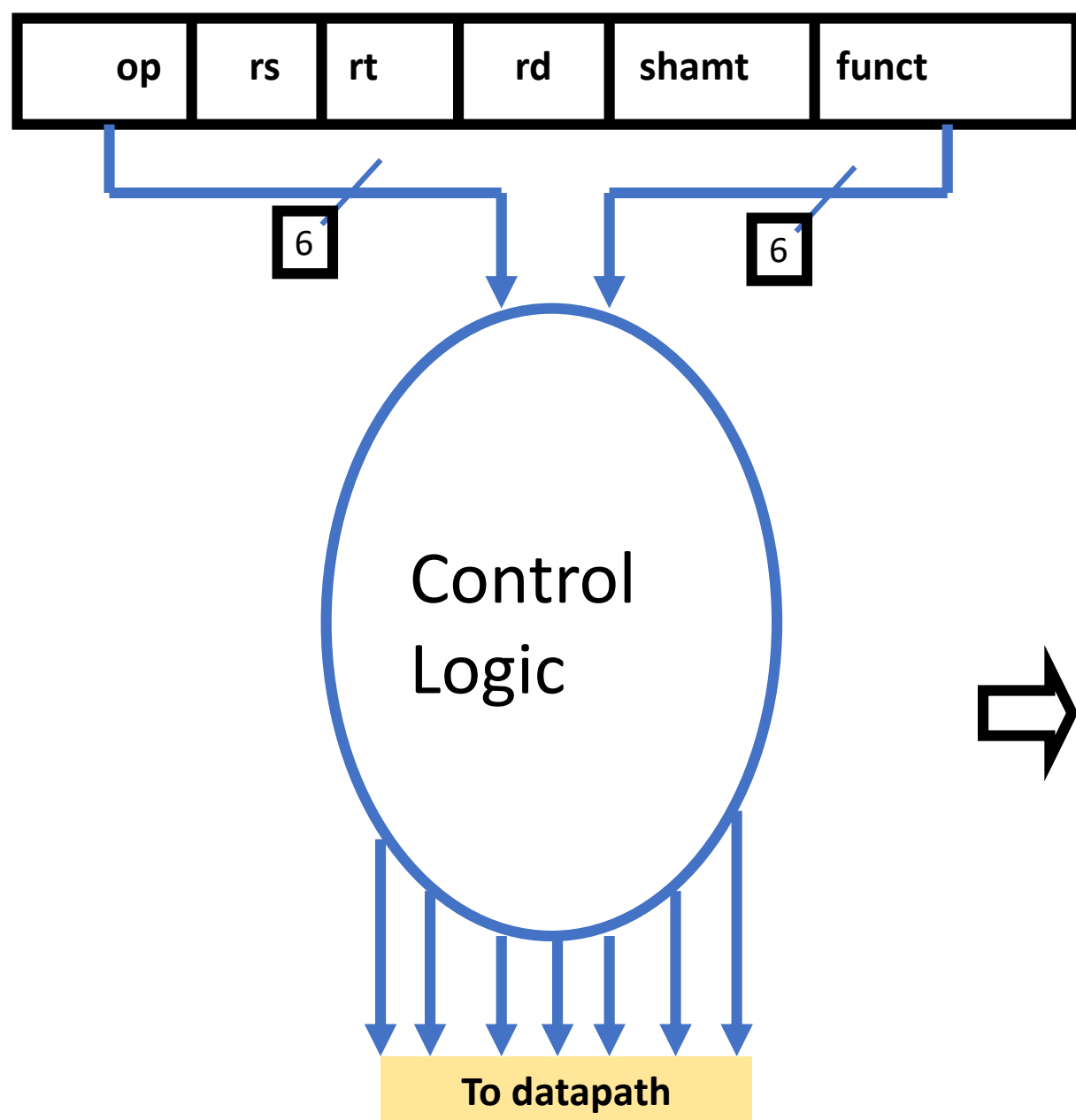
MIPS SINGLE CYCLE DATAPATH



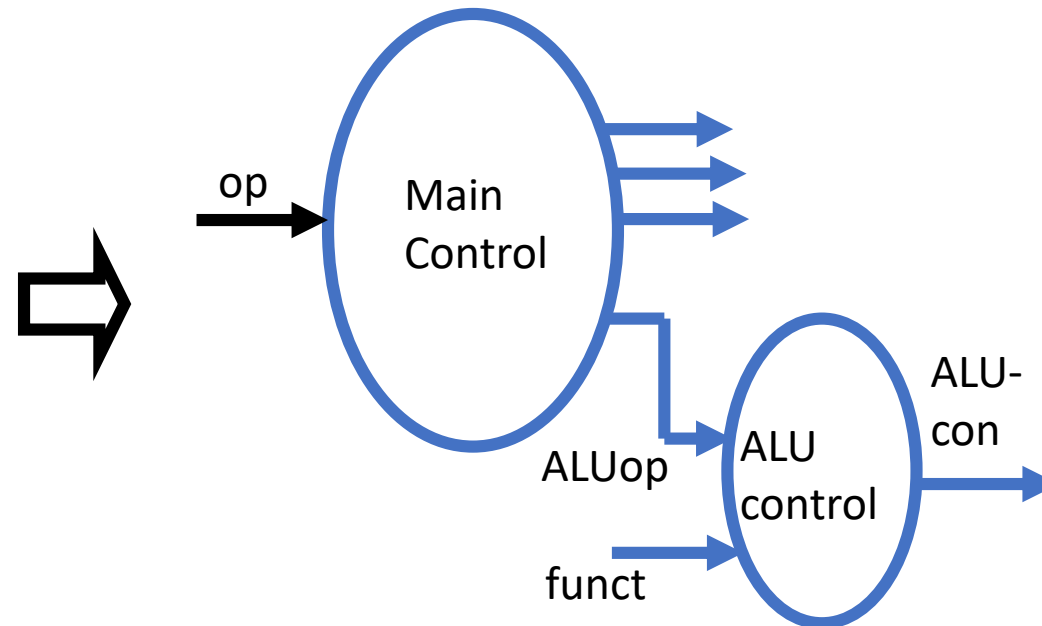
PROCESSOR = DATAPATH + CONTROL

- **Single-Cycle Design:** everything happens in one clock cycle $\Rightarrow$ until next falling/rising edge of a clock, processor acts as one big combinational circuit!!!
 - Each Instruction takes exactly one clock cycle to complete their operation
 - Datapath units are used only once per cycle
 - Writable state gets updated at end of cycle
- **Control** is just a combinational circuit (i.e. output is just a function of inputs)
 - Inputs: the current instruction! (`opcode`, `funct` control everything)
 - Outputs: control points in datapath
- What must be “controlled”?
 - Multiplexors (Muxes)
 - ALU (which operation?)
 - Writable state elements: Register File, Data Memory (Dmem)

DEFINING CONTROL



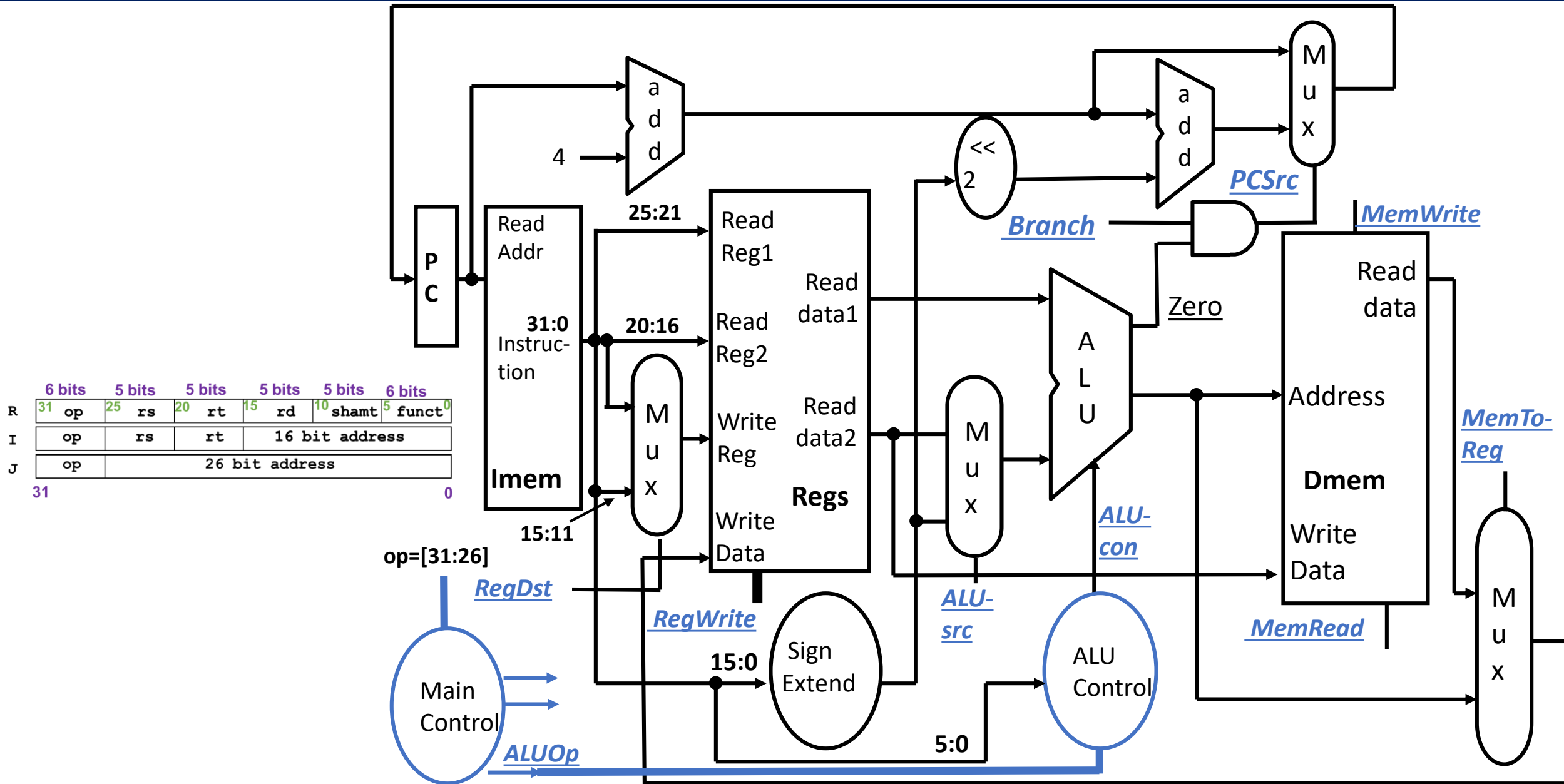
- “funct field” is present only in R-format instruction → funct controls ALU only.
- To simplify control, Let us define Main, ALU control separately – using multiple levels will also increase speed – important optimization technique



CONTROL SIGNALS

Signal name	Effect when deasserted	Effect when asserted
RegDst	The register destination number for the Write register comes from the rt field (bits 20:16).	The register destination number for the Write register comes from the rd field (bits 15:11).
RegWrite	None.	The register on the Write register input is written with the value on the Write data input.
ALUSrc	The second ALU operand comes from the second register file output (Read data 2).	The second ALU operand is the sign-extended, lower 16 bits of the instruction.
PCSrc	The PC is replaced by the output of the adder that computes the value of PC + 4.	The PC is replaced by the output of the adder that computes the branch target.
MemRead	None.	Data memory contents designated by the address input are put on the Read data output.
MemWrite	None.	Data memory contents designated by the address input are replaced by the value on the Write data input.
MemtoReg	The value fed to the register Write data input comes from the ALU.	The value fed to the register Write data input comes from the data memory.

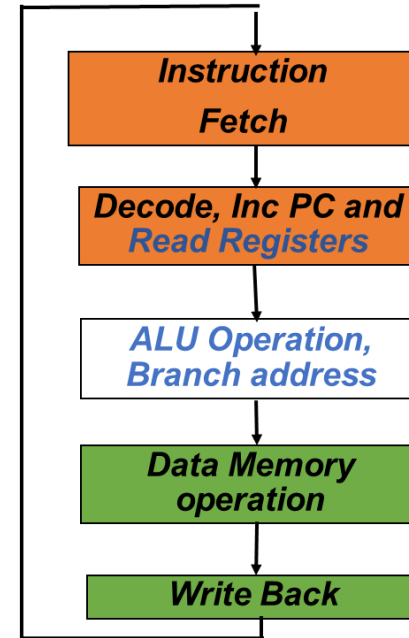
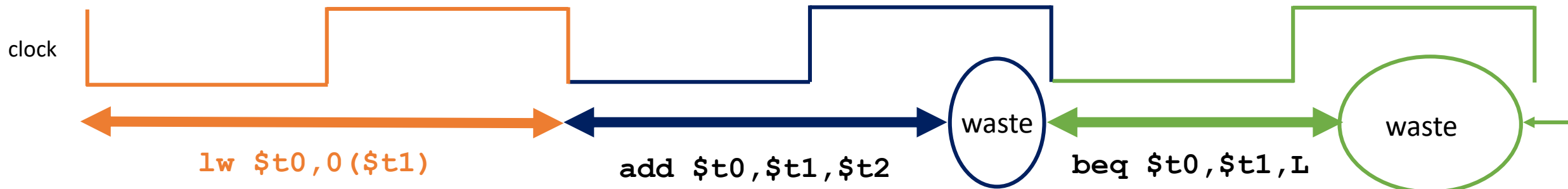
SINGLE-CYCLE MIPS (MINI-REFERENCE) CPU



SINGLE-CYCLE MACHINE: APPRAISAL

- All instructions complete in one clock cycle (CPI = 1)
- Some instructions take more steps than others
 - lw is most expensive (5 steps), vs. sw (4 steps)
 - R-type instructions take 4 cycles
 - 3 steps for beq
- Clock cycle must cover longest instruction $\Rightarrow$ inefficient
 - suppose mul is added?
 - 32-shift/add steps $\Rightarrow$ would delay every other instruction

Single-cycle Implementation



lw \$t0, 0(\$t1)
add \$t0, \$t1, \$t2
beq \$t0, \$t1, Lab

EXAMPLE QUESTIONS

- Assume 2ns for instruction/data memory, 1ns for decode/register read, 2ns for ALU and 1 ns for register write.

Single-cycle datapath clock period = ?

I-type	IF	ID	EX	Mem	WB	Total
R-type	2	1	2	0	1	6
LW (I)	2	1	2	2	1	8
SW (I)	2	1	2	2	0	7
BR (I)	2	1	2	0	0	5

- Assume memory read takes 6ns, memory write takes 10ns, register read/write takes 2ns, ALU operation takes 2ns and rest 1ns. Compute the best possible clock frequency for this single cycle MIPS processor.

Clock Frequency = ?

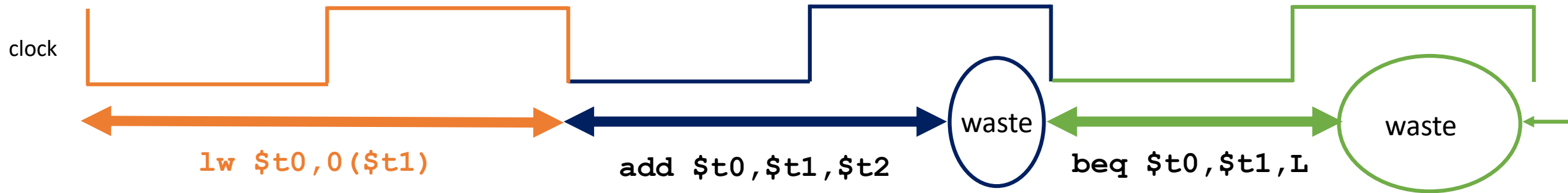
MULTICYCLE IMPLEMENTATION

- We want more efficient implementation
- Each **step** will take one clock cycle (not each instruction) [CPI > 1]
⇒ shorter clock cycle: cycle time **constrained by longest step, not longest insn.**
- simpler instructions take fewer cycles
⇒ higher overall performance
- complex control: [finite state machine](#)
- Versatile (can extend for new instructions: add3, swap, etc.)

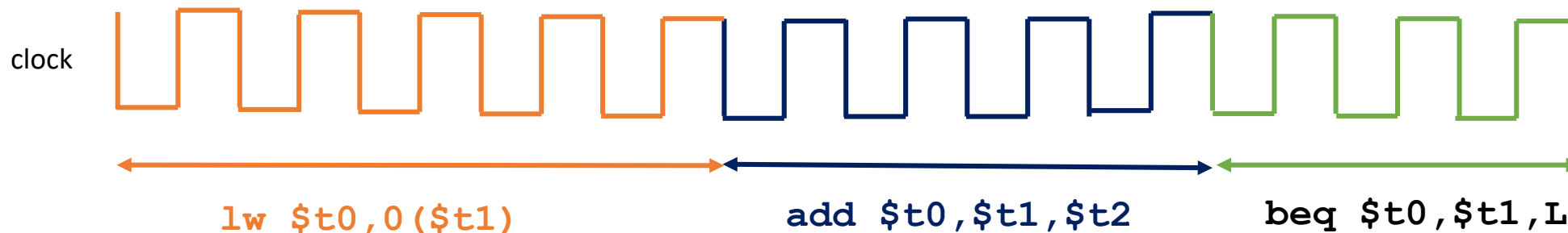
CLOCKING: SINGLE-CYCLE VS. MULTICYCLE PROCESSOR

```
lw $t0, 0($t1)
add $t0, $t1, $t2
beq $t0, $t1, Lab
```

Single-cycle Implementation



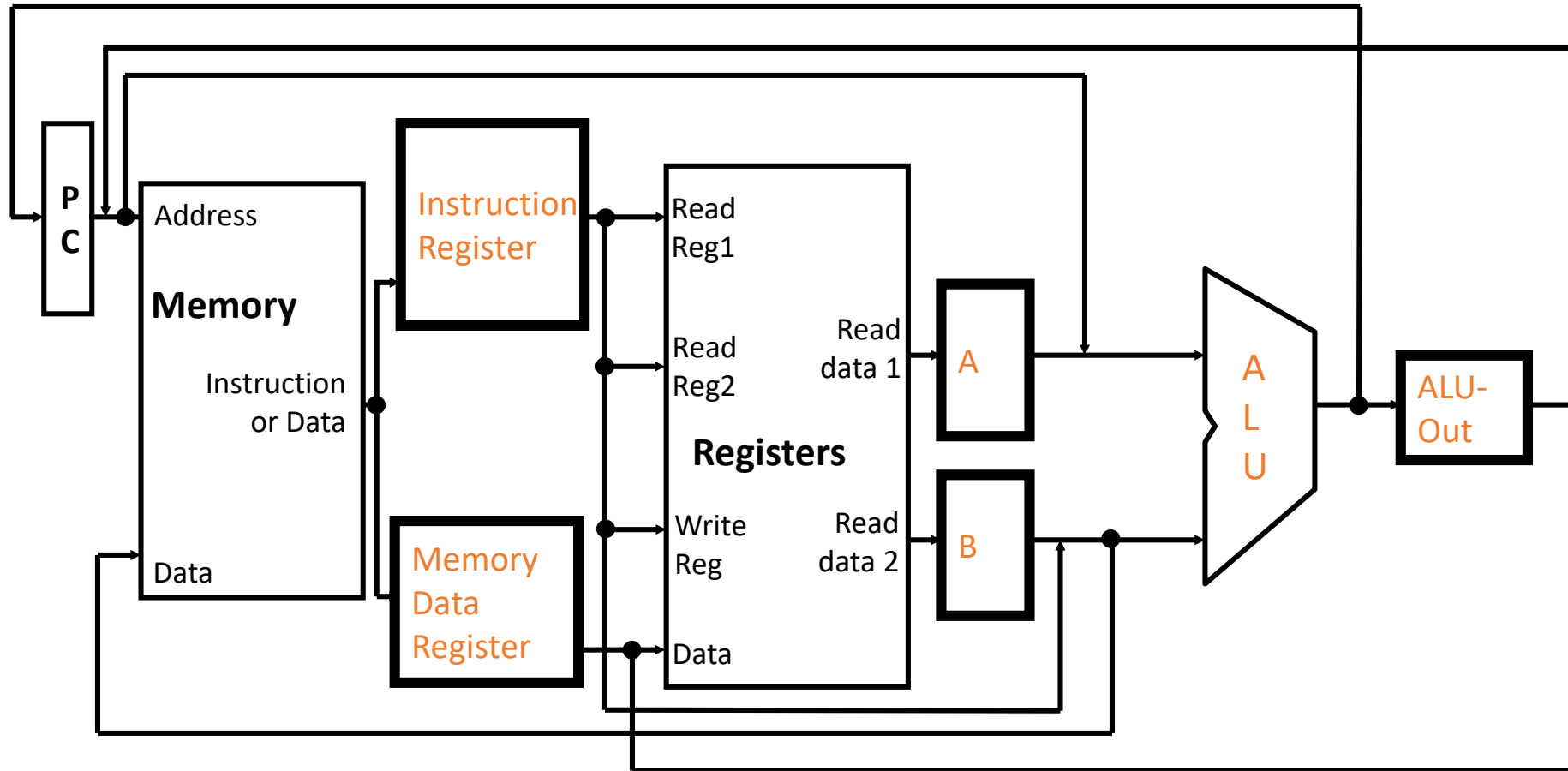
Multicycle Implementation



- Multicycle Implementation → less waste & higher performance (*most often*)

MULTICYCLE DATAPATH (OVERVIEW)

MIPS-mini Multicycle Version



- *One ALU (no extra adders)*
- *One Memory (no separate Imem, Dmem)*
- *New Temporary Registers ("clocked"/require clock input)*

MULTICYCLE IMPLEMENTATION

- Datapath changes
 - one memory: both instructions and data (because can access on separate *steps*)
 - one ALU (eliminate extra adders)
 - extra “invisible” registers to capture intermediate (per-step) datapath results
- Controller changes
 - controller must fire control lines in *correct sequence and correct time*
⇒ controller must remember current execution step, advance to next step

MULTICYCLE INSTRUCTION EXECUTION

- All instructions execute in 3-5 [cycles](#)
 - 3 cycles: `beq`
 - 4 cycles: R-type, `sw`
 - 5 cycles: `lw`
- 1st Cycle: fetch instruction, $PC=PC+4$
- 2nd Cycle: decode, fetch registers, branch target
- 3rd Cycle: execute ALU operation/compute address/branch
- 4th Cycle: access memory/complete R-type
- 5th Cycle: store content from memory to register (`load operations`).

- If we understand the instructions...
We can build a simple processor!
- If instructions take different amounts of time, multi-cycle may be better!
- Datapath implementation using:
 - Combinational logic for arithmetic
 - State holding elements to remember bits.
- Control implementation using:
 - Combinational logic for single-cycle implementation
 - Finite state machine for multi-cycle implementation

RECAP: SINGLE CYCLE VS MULTICYCLE PROCESSOR DESIGN

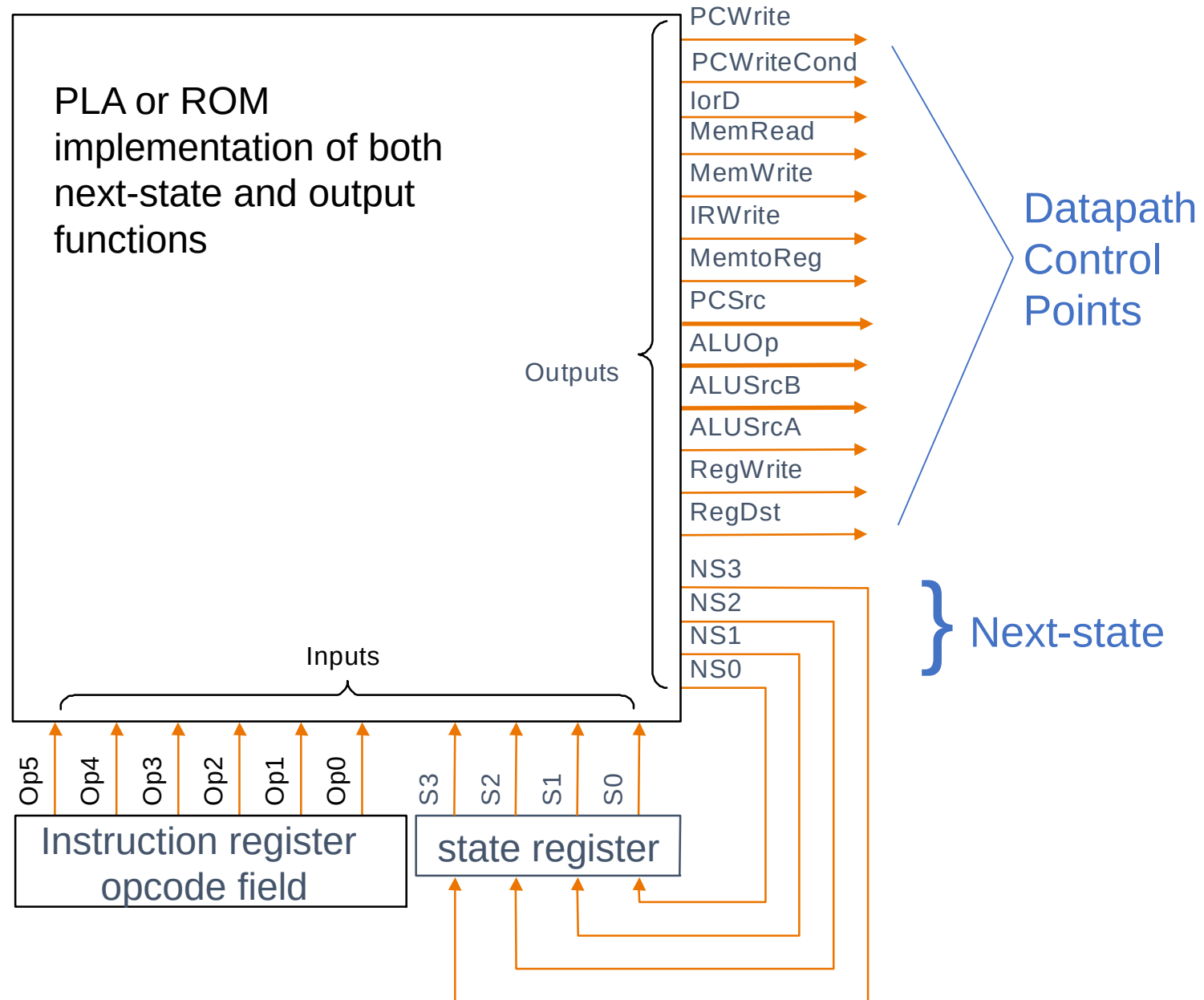
Single Cycle Design

- All instructions execute in single cycle.
 - CPI = 1
 - Instruction execution may still be carried out in multiple states (3/4/5), but all within one cycle.
- Clock cycle is determined by the longest execution time of the instruction.
- Different Instruction types need different amount of time
 - *Results in wastage of portion of cpu cycles*
- Control unit is simple and can generate all control signals for entire instruction at once.

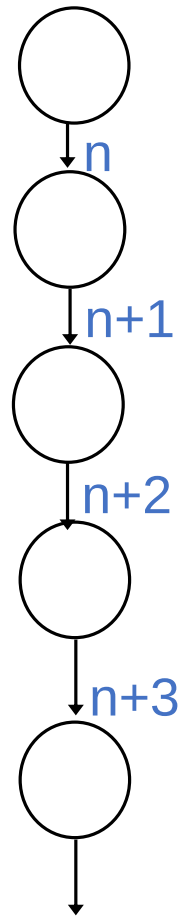
Multi-Cycle Design

- All instructions execute in 3-5 cycles.
 - CPI = 3 cycles: `beq`
 - CPI = 4 cycles: `R-type`, `sw`
 - CPI = 5 cycles: `lw`
- Clock cycle is determined by the critical steps in the instruction execution.
- Allows reuse of some hardware blocks.
- Control unit is complex, needs to track current state (FSM) to generate appropriate control signals.
- Demands extra memory elements to keep track of intermediate state.

CONTROL PROCESSING IN MULTI-CYCLE PROCESSOR



OBSERVATION ABOUT REAL MACHINES



- Machine Language: next instruction to be executed is usually implied
 - PC register determines instruction
 - next instruction always at PC+4 (unless branch or jump)
- FSM Controller: often only one exit arc from current state to next state
- Suppose borrow idea from Machine Language, represent each control step as some kind of “instruction”?
- Leads to Microprogrammed Control

MICRO-PROGRAMMED CONTROL

- In microprogrammed control, FSM states become microinstructions of a microprogram (“microcode”)
 - one FSM state=one microinstruction
 - usually represent each micro-instruction textually, like an assembly instruction
- FSM current state register becomes the microprogram counter (micro-PC)
 - normal sequencing: add 1 to micro-PC to get next micro-instruction
 - microprogram branch: separate logic determines next microinstruction

MICROPROGRAMMING VS HARDWIRED CONTROL



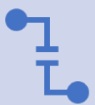
Microprogramming offers flexibility for design and architectural changes.

The control memory (ROM) can be reprogrammed or replaced.



Microprogramming is slow.

Needs access to control memory every clock cycle; memory access is slow.



Hardwired control is difficult to design for complex set architecture.

Once the control is designed, no further change is possible.



Hardwired control is fast.

Cycle time depends on combinational logic delay of the control unit; which is $\ll$ memory access time.